

Programação Orientada a Objetos 2

Encontro 6

Coleções e generics; Uso de coleções e generics

Aulas previstas

Aula	Data
Revisão Avaliação 1	17/09/25
Avaliação 1	24/09/25

Introdução

- **Coleções:** estruturas de dados que armazenam múltiplos elementos.
- **Generics:** mecanismo para criar classes, interfaces e métodos parametrizados por tipo.
- **Importância:** facilitam o gerenciamento de dados, promovem reuso e aumentam a segurança de tipos.
- **Aplicações:** listas, conjuntos, mapas, filas, pilhas, etc.
- **Java Collections Framework:** conjunto de classes e interfaces para manipulação de coleções.

Generics em Java

Generics são um mecanismo que permite criar classes, interfaces e métodos parametrizados por tipo:

- **Type Safety:** detecta erros de tipo em tempo de compilação
- **Eliminação de casts:** não precisa fazer casting explícito
- **Reutilização de código:** mesmo código funciona com diferentes tipos

Características:

- Avançado: Wildcards (?, ? extends, ? super)
- Não permite tipos primitivos diretamente

Exemplo prático – Generics em Java

```
import java.util.*;

// Classe genérica
class Caixa<T> {
    private T conteudo;

    public void colocar(T item) { this.conteudo = item; }
    public T retirar() { return conteudo; }
}
```

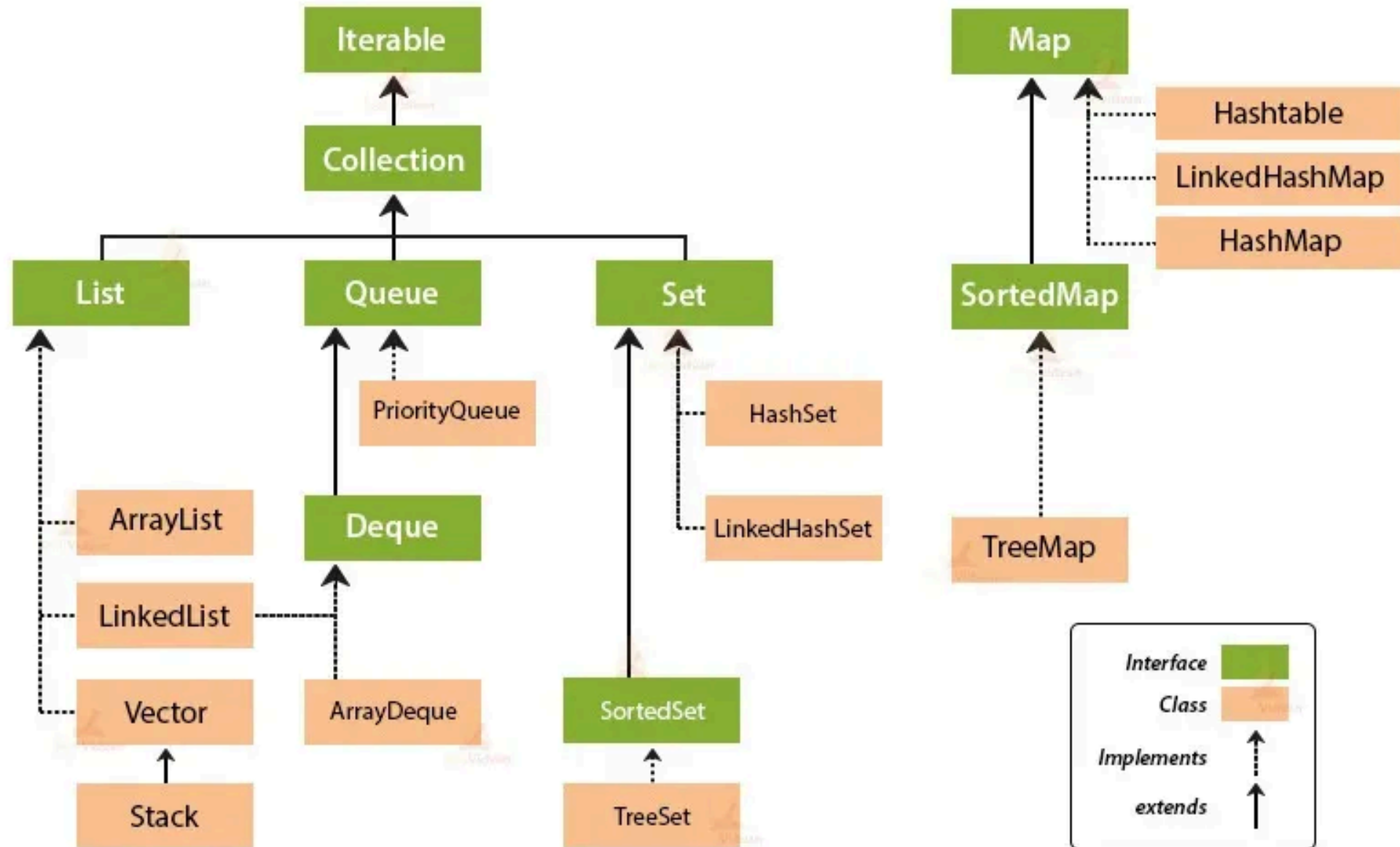
```
import java.util.*;

public class ExemploGenerics {
    public static void main(String[] args) {
        // Uso da classe genérica
        Caixa<String> caixaTexto = new Caixa<>();
        caixaTexto.colocar("Olá");
        String texto = caixaTexto.retirar(); // Sem cast

        // Coleções com generics
        List<String> nomes = new ArrayList<>();
        nomes.add("João");
        // nomes.add(123); // Erro de compilação

        // Wildcards
        List<Integer> numeros = new ArrayList<>();
        // numeros.add("4"); // Erro de compilação
        numeros.add(1);
        numeros.add(2);
        numeros.add(3);
    }
}
```

Collection Framework Hierarchy in Java



Iterable

- **Iterable:** raiz da hierarquia de coleções, permite iteração sobre elementos.

Todo objeto que implementa Iterable pode ser percorrido com um iterator ou por um for-each loop.

```
Iterable<Elemento> it = ...;  
for (Elemento e : it) {  
    // usar  
}
```

Collection

- **Collection:** estende Iterable, representa um grupo de objetos (elementos).
Define operações básicas como adicionar, remover, verificar tamanho e limpar a coleção.

```
Collection<Elemento> colecao = ...;  
colecao.add(new Elemento());  
colecao.remove(new Elemento());  
int tamanho = colecao.size();  
colecao.clear();
```


List

List é uma interface da Java Collections Framework que representa uma coleção ordenada:

- **Permite elementos duplicados** na coleção
- **Mantém a ordem de inserção** dos elementos
- **Acesso por índice** - permite acesso direto aos elementos através de posição

Características:

- Redimensionamento dinâmico
- Métodos para adicionar, remover e buscar elementos
- Suporte a iteração sequencial
- Permite valores nulos

Exemplo prático – List

```
import java.util.*;

public class ExemploList {
    public static void main(String[] args) {
        List<String> nomes = new ArrayList<>();

        // Adicionando elementos
        nomes.add("João");
        nomes.add("Maria");
        nomes.add("Pedro");
        nomes.add("Maria"); // Permite duplicatas

        // Operações principais
        System.out.println("Primeiro: " + nomes.get(0));
        System.out.println("Tamanho: " + nomes.size());

        nomes.set(1, "Ana");           // Substituir
        nomes.remove("Pedro");         // Remover por valor
        nomes.remove(0);               // Remover por índice

        // Busca
        boolean tem = nomes.contains("Maria");
        int indice = nomes.indexOf("Maria");

        System.out.println("Contém Maria: " + tem);
        System.out.println("Índice: " + indice);
    }
}
```

Ordenação em List

- **Collections.sort(list)**: ordena a lista em ordem natural (alfabética, numérica).
- **Comparator**: permite definir critérios personalizados de ordenação.

```
List<String> nomes = new ArrayList<>();  
nomes.add("João");  
nomes.add("Maria");  
nomes.add("Pedro");  
Collections.sort(nomes); // Ordem alfabética
```

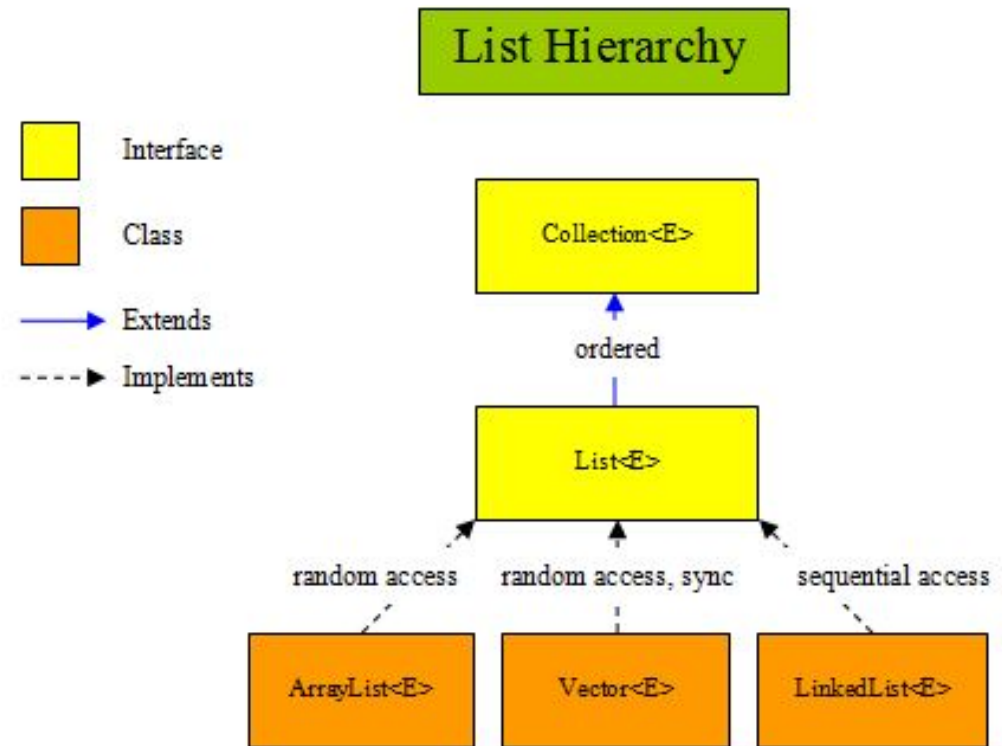
Ordenação em List

- **Collections.sort(list):** ordena a lista em ordem natural (alfabética, numérica).
- **Comparator:** permite definir critérios personalizados de ordenação.

```
List<Aluno> alunos = new ArrayList<>();  
alunos.add(new Aluno("João", 8.5));  
alunos.add(new Aluno("Maria", 9.0));  
alunos.add(new Aluno("Pedro", 7.5));  
Collections.sort(alunos); // Requer que Aluno implemente Comparable<Aluno>
```

Implementações

- **Implementações principais:**
ArrayList, LinkedList, Vector.



Operation \ List	LinkedList	ArrayList
get(int index)	$O(n/4)$ average	$O(1)$
add(E element)	$O(1)$	$O(1)$ amortized $O(n)$ worst-case
add(int index, E element)	$O(n/4)$ average $O(1)$ if index is 0	$O(n/2)$ average
remove	$O(n/4)$ average	$O(n/2)$ average
Iterator.remove()	$O(1)$	$O(n/2)$ average
ListIterator.add(E element)	$O(1)$	$O(n/2)$ average

Comparable

- **Comparable:** interface que define a ordem natural dos objetos.
- **compareTo(T o):** método que compara o objeto atual com outro do mesmo tipo
- Retorna:
 - Negativo se o atual é menor que o outro
 - Zero se são iguais
 - Positivo se o atual é maior que o outro

```
class Aluno implements Comparable<Aluno> {  
    private String nome;  
    private double nota;  
  
    public Aluno(String nome, double nota) {  
        this.nome = nome;  
        this.nota = nota;  
    }  
  
    @Override  
    public int compareTo(Aluno outro) {  
        return String.compare(this.nome, outro.nome);  
    }  
}
```


Comparator

- **Comparator**: interface funcional para definir múltiplos critérios de ordenação.
- **compare(T o1, T o2)**: método que compara dois objetos.
- Pode ser usado para ordenações temporárias sem alterar a ordem natural.

```
List<Aluno> alunos = new ArrayList<>();
alunos.add(new Aluno("João", 8.5));
alunos.add(new Aluno("Maria", 9.0));
alunos.add(new Aluno("Pedro", 7.5));
Collections.sort(alunos, new Comparator<Aluno>() {
    @Override
    public int compare(Aluno a1, Aluno a2) {
        return Double.compare(a1.getNota(), a2.getNota());
    }
});
```

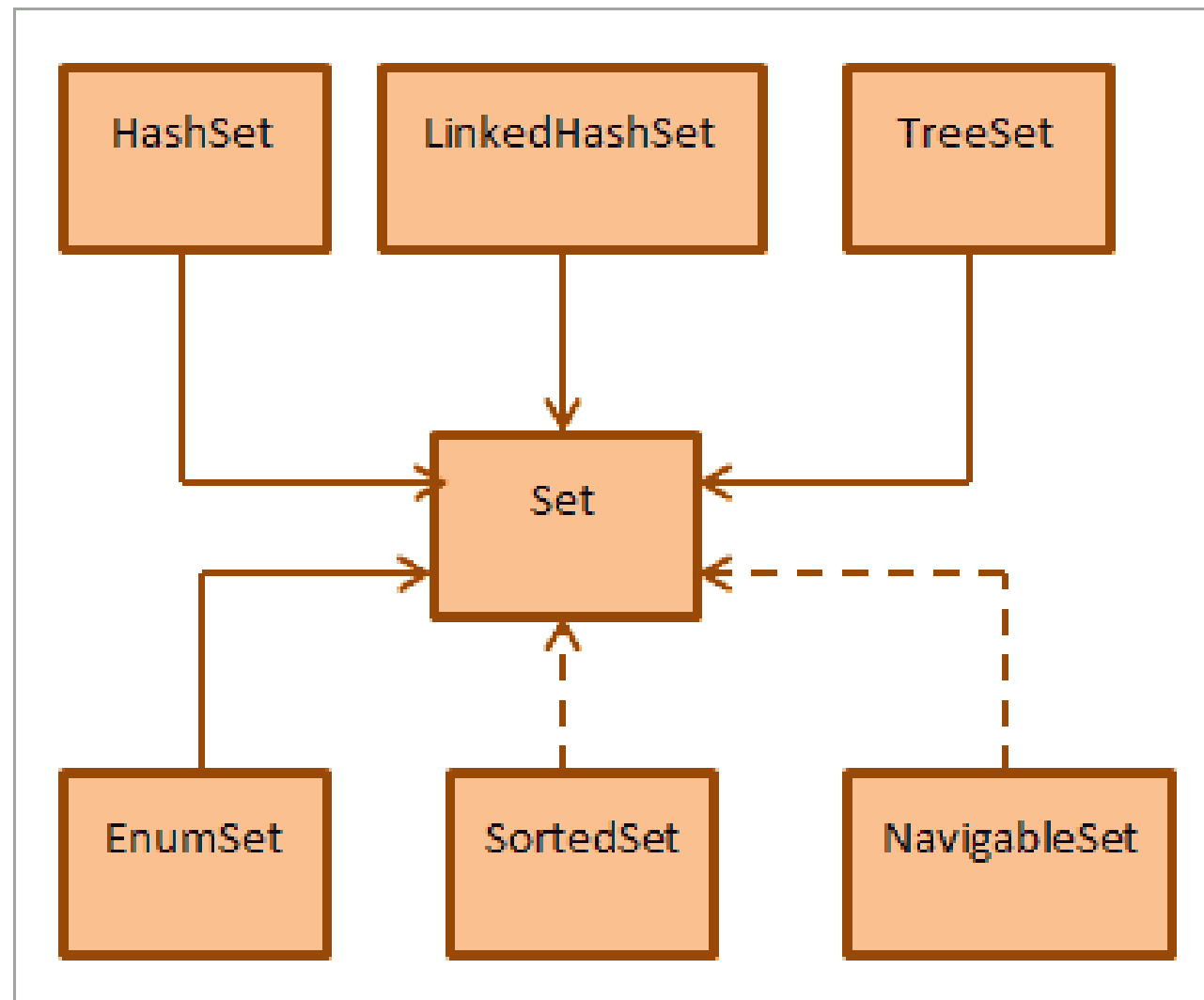
Set

Set é uma interface da Java Collections Framework que representa uma coleção única:

- **Não permite elementos duplicados** - cada elemento é único
- **Não mantém ordem específica** (exceto implementações ordenadas)
- **Implementações principais:** HashSet, LinkedHashSet, TreeSet
- **Baseado na matemática de conjuntos**
- **Operações de conjunto:** união, interseção, diferença
- **Não permite acesso por índice**

Características:

- HashSet: melhor performance, sem ordem
- LinkedHashSet: mantém ordem de inserção
- TreeSet: elementos ordenados naturalmente
- Operações de conjunto (união, interseção, diferença)



Exemplo prático – Set

```
import java.util.*;

public class ExemploSet {
    public static void main(String[] args) {
        Set<String> cores = new HashSet<>();
        Set<String> ordenadas = new TreeSet<>();

        // Adicionando (duplicatas ignoradas)
        cores.add("Azul");
        cores.add("Vermelho");
        cores.add("Azul"); // Será ignorado

        System.out.println("Únicas: " + cores.size()); // 2

        // TreeSet mantém ordem
        ordenadas.addAll(cores);
        System.out.println("Ordenadas: " + ordenadas);
    }
}
```

```
import java.util.*;

public class ExemploSet {
    public static void main(String[] args) {

        // Operações de conjunto
        Set<String> set1 = Set.of("A", "B", "C");
        Set<String> set2 = Set.of("B", "C", "D");

        Set<String> uniao = new HashSet<>(set1);
        uniao.addAll(set2); // União

        Set<String> intersecao = new HashSet<>(set1);
        intersecao.retainAll(set2); // Interseção

        System.out.println("União: " + uniao);
        System.out.println("Interseção: " + intersecao);
    }
}
```

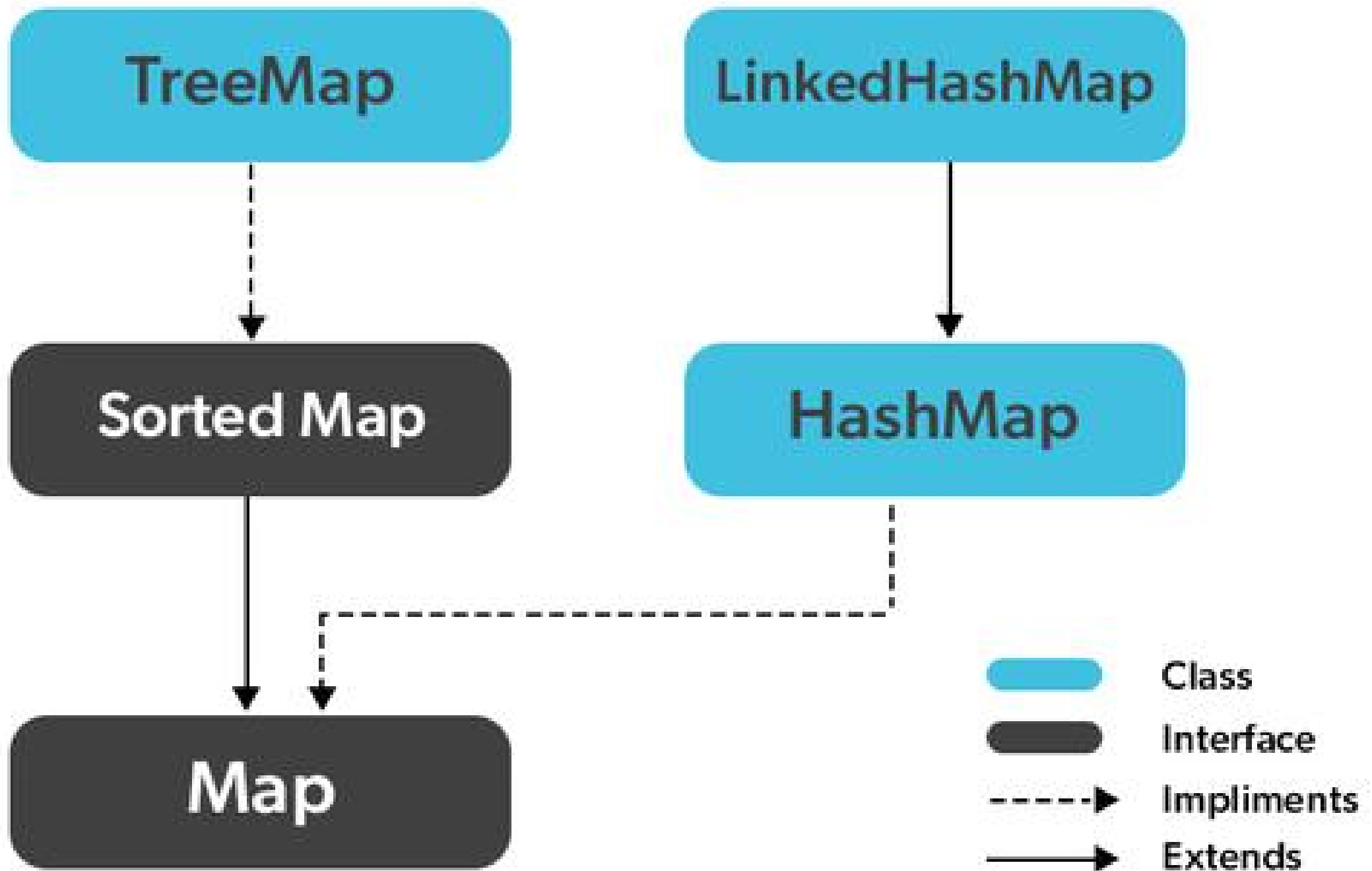
Map

Map é uma interface que representa uma coleção de pares chave-valor:

- **Associa chaves únicas a valores** - cada chave mapeia para um valor
- **Não permite chaves duplicadas** - mas valores podem se repetir
- **Implementações principais:** HashMap, LinkedHashMap, TreeMap
- **Não é uma Collection** - é uma interface separada

Características:

- HashMap: melhor performance, sem ordem
- LinkedHashMap: mantém ordem de inserção
- TreeMap: chaves ordenadas naturalmente
- Acesso rápido por chave ($O(1)$ para HashMap)



Exemplo prático – Map

```
import java.util.*;

public class ExemploMap {
    public static void main(String[] args) {
        Map<String, Integer> idades = new HashMap<>();

        // Adicionando pares chave-valor
        idades.put("João", 25);
        idades.put("Maria", 30);
        idades.put("João", 26);    // Atualiza valor existente

        // Acessando valores
        System.out.println("João: " + idades.get("João"));

        // Verificações
        boolean temPedro = idades.containsKey("Pedro");
        boolean tem30 = idades.containsValue(30);

        // Iterando sobre Map
        for (Map.Entry<String, Integer> entry : idades.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }

        // Operações úteis
        idades.remove("Maria");
        int padrao = idades.getOrDefault("Ana", 0);

        System.out.println("Ana (padrão): " + padrao);
    }
}
```

Exercício

- Implementar uma ordenação natural para a classe Produto (pelo nome do produto).
- Implementar uma ordenação personalizada para a classe Produto (pela quantidade em estoque).
- Criar uma lista de produtos, adicionar alguns produtos e ordenar usando ambas as ordenações.

Referências

- Page-Jones, M. *Fundamentos do Desenho Orientado a Objeto com UML*. Makron Books.
- Larman, C. *Utilizando UML e Padrões*. Saraiva.
- Ementa da disciplina.